

Fundamentos de Computadores

1º curso del Grado en Ingeniería Informática

Tema 1

Introducción a los sistemas digitales



*Departamento de Ingeniería Electrónica,
de Sistemas Informáticos y Automática*

Tema 1. Introducción a los sistemas digitales

Definición de sistema

Se puede definir un **sistema** como un conjunto de partes interrelacionadas que cooperan entre sí para llevar a cabo una determinada tarea.

Más concretamente, un sistema puede entenderse como cualquier proceso que lleve a cabo una transformación de señales. Así pues, todo sistema recibe una excitación procedente desde el exterior (**señales de entrada**) y proporciona una respuesta (**señales de salida**) como resultado del proceso en base a un **comportamiento** predefinido.



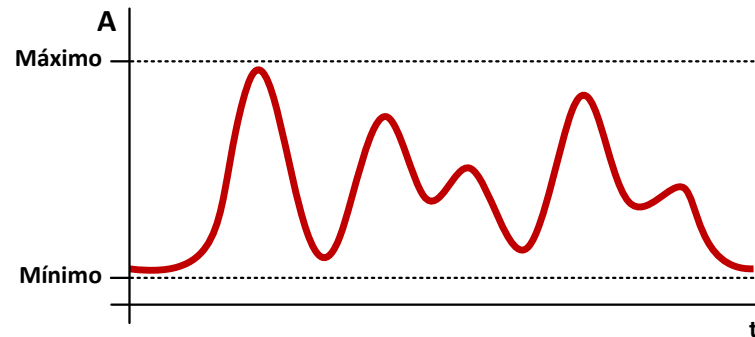
En el entorno que nos rodea podemos encontrar ejemplos de sistemas de diversa naturaleza, asociados a una amplia variedad de campos, tales como los siguientes:

- Comunicaciones.
- Aeronáutica y astronáutica.
- Diseño de circuitos.
- Acústica.
- Sismología.
- Ingeniería biomédica.
- Sistemas de distribución y de generación de energía.
- Control de procesos químicos.
- Procesado de señales.

Tema 1. Introducción a los sistemas digitales

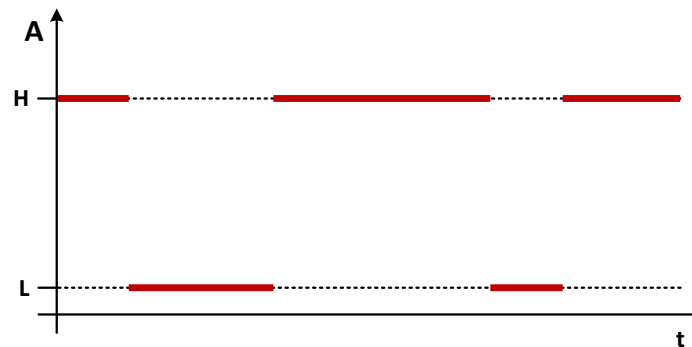
Sistemas analógicos y digitales

Una variable **analógica** o **continua** es aquella cuya amplitud (A) varía en el tiempo de manera continua, dentro del rango comprendido entre los valores mínimo y máximo que la misma puede adoptar.



Los **sistemas analógicos** son aquellos que procesan variables de naturaleza analógica o continua.

Por el contrario, una variable **digital** o **discreta** es aquella cuya amplitud (A) sólo puede adoptar un número finito de valores. Debido a la relativa facilidad que presenta la fabricación de componentes electrónicos con dos estados de funcionamiento claramente diferenciados, las variables digitales que se emplean en la práctica poseen solamente dos valores, un nivel alto o **H** (*High*) y un nivel bajo o **L** (*Low*).



Los **sistemas digitales** son aquellos que procesan variables de naturaleza digital o discreta.

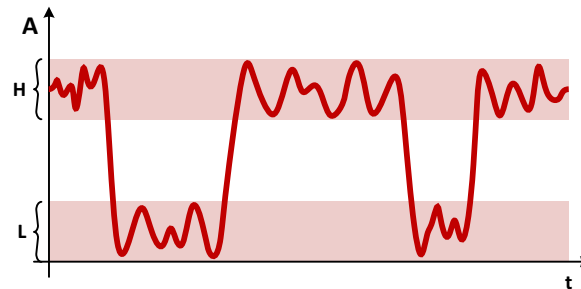
Tema 1. Introducción a los sistemas digitales

Lógica positiva y negativa

La variable digital mostrada en la figura anterior es una variable **ideal**, por dos razones:

- Su amplitud se mantiene constante para los niveles alto y bajo.
- Los cambios o transiciones entre ambos niveles se producen de manera instantánea.

En los sistemas utilizados en la práctica, las variables digitales se representan mediante señales de naturaleza analógica, normalmente con tensiones eléctricas, por lo que las variables digitales **reales** presentan un comportamiento temporal similar al mostrado en la siguiente gráfica.



- Por una parte se observa que los niveles H y L que puede adoptar la señal no son valores concretos, sino rangos de valores.
- Por otro lado, se aprecia que las transiciones entre ambos rangos no son instantáneas, sino que requieren un cierto tiempo, por lo que durante ese periodo la señal no posee el valor H ni el L, y por tanto no es válida (no se puede utilizar).

Las señales digitales se emplean en los sistemas reales para representar los dos valores del sistema binario (el **0** y el **1**).

Según la correspondencia que se establezca entre los niveles H y L que adoptan las señales y los valores 0 y 1 se pueden utilizar dos tipos de lógica: **lógica positiva** y **lógica negativa**.

Lógica positiva

L ↔ 0
H ↔ 1

Lógica negativa

L ↔ 1
H ↔ 0

Por lo general se suele usar la lógica positiva, y será éste el convenio que se empleará en adelante en esta asignatura.

Por último, indicar que una variable que puede adoptar solamente dos valores, el **0** y el **1**, recibe el nombre de **bit**.

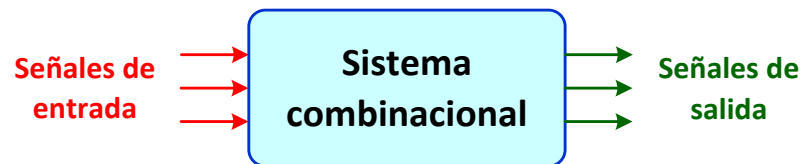
Tema 1. Introducción a los sistemas digitales

Tipos de sistemas digitales

Los sistemas digitales se pueden clasificar en dos tipos: **sistemas combinacionales** y **sistemas secuenciales**.

Sistemas combinacionales

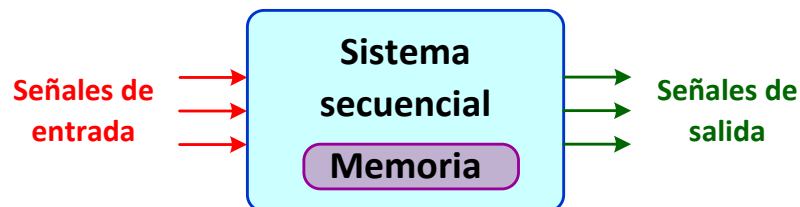
Un **sistema combinacional** es un sistema digital en el que el estado de sus salidas en un instante determinado sólo depende de los valores presentes en sus entradas en ese mismo instante, siendo independiente de la secuencia de entradas que se haya aplicado al sistema con anterioridad.



Sistemas secuenciales

Un **sistema secuencial** es un sistema digital en el que el estado de sus salidas en un instante determinado depende, además de los valores presentes en sus entradas en ese mismo instante, de la secuencia de entradas que se haya aplicado al sistema con anterioridad.

Por tanto, un sistema secuencial tiene de algún modo la capacidad de recordar, es decir posee **memoria**.



Los **computadores**, dispositivos que dan nombre a esta asignatura, son sistemas digitales de tipo secuencial

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales

Un sistema de numeración es un conjunto de símbolos que se emplean para representar información numérica.

Un **sistema de numeración posicional** es aquel sistema de numeración en el que cualquier número se representa mediante una cadena de dígitos, cada uno de los cuales está afectado por un factor de escala que depende de la posición que éste ocupa dentro de la cadena.

Se denomina **base** o **raíz** de un sistema posicional al número de símbolos diferentes que se utilizan para representar las cantidades en dicho sistema. Así pues, los sistemas posicionales se caracterizan por el valor de su base, denotándose como sistemas de base 2, 3, ..., 8, 9, 10, etc. Por otro lado, se denomina **alfabeto** al conjunto concreto de símbolos empleados para representar las cantidades en el sistema posicional.

En los sistemas digitales se utilizan, básicamente, los cuatro sistemas de numeración siguientes.

Nombre del sistema posicional	Base	Alfabeto
Sistema decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Sistema binario	2	0, 1
Sistema octal	8	0, 1, 2, 3, 4, 5, 6, 7
Sistema hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

En un sistema posicional de base **b**, un número cualquiera **N** puede representarse mediante un polinomio de potencias de dicha base, multiplicadas por coeficientes que también pertenecen al sistema:

$$N_b = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + \dots + a_i \cdot b^i + \dots + a_0 \cdot b^0 + a_{-1} \cdot b^{-1} + \dots + a_{-p} \cdot b^{-p} = \sum_{i=-p}^n a_i \cdot b^i$$

En la expresión anterior, **a_i** es un símbolo perteneciente al sistema de numeración ($0 \leq a_i < b$), **n+1** es el número de dígitos enteros del número y **p** es el de dígitos fraccionarios.

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales

El número de cantidades que se pueden representar en un sistema posicional de base **b** mediante una combinación de **n** dígitos viene expresado por la siguiente expresión:

$$b^n$$

Por tanto, para representar **m** cantidades diferentes, el número **n** de dígitos necesario será el menor entero que cumpla la expresión:

$$n \geq \log_b m$$

Según esto, para representar los números del 0 al 15 ($m = 16$), en los sistemas decimal, binario, octal y hexadecimal se necesitan **2**, **4**, **2** y **1** dígitos, respectivamente.

Sistema decimal	Sistema binario	Sistema octal	Sistema hexadecimal
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales (Conversión entre las distintas bases de numeración)

Conversión de una base cualquiera a decimal

Para realizar la conversión a decimal de una cantidad expresada en otra base cualquiera se emplea el **método polinómico**, cuyas principales características son las siguientes:

- Se expresa el número a convertir mediante su polinomio equivalente en la base de origen.
- Se evalúa el polinomio usando la aritmética de la base de destino (**base 10**).
- La conversión de las partes entera y fraccionaria de la cantidad se realiza al mismo tiempo.
- Los exponentes del polinomio en la parte entera son positivos y en la fraccionaria son negativos.

Ejemplos:

$$\begin{matrix} 4 & 3 & 2 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \end{matrix}_2 = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 16 + 2 + 1 = 19_{10}$$

$$\begin{matrix} 0 & -1 & -2 & -3 & -4 \\ 0 & .1 & 0 & 0 & 1 \end{matrix}_2 = 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 0.5 + 0.0625 = 0.5625_{10}$$

$$\begin{matrix} 5 & 4 & 3 & 2 & 1 & 0 & -1 & -2 & -3 & -4 \\ 1 & 0 & 1 & 0 & 1 & 1 & .0 & 1 & 1 & 1 \end{matrix}_2 = 1 \times 2^5 + 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} = 32 + 8 + 2 + 1 + 0.25 + 0.125 + 0.0625 = 43.4375_{10}$$

$$\begin{matrix} 4 & 3 & 2 & 1 & 0 \\ 2 & 7 & 6 & 3 & 4 \end{matrix}_8 = 2 \times 8^4 + 7 \times 8^3 + 6 \times 8^2 + 3 \times 8^1 + 4 \times 8^0 = 8192 + 3584 + 384 + 24 + 4 = 12188_{10}$$

$$\begin{matrix} 2 & 1 & 0 & -1 & -2 & -3 \\ 7 & 1 & 5 & .3 & 6 & 2 \end{matrix}_8 = 7 \times 8^2 + 1 \times 8^1 + 5 \times 8^0 + 3 \times 8^{-1} + 6 \times 8^{-2} + 2 \times 8^{-3} = 448 + 8 + 5 + 0.375 + 0.09375 + 0.00390625 = 461.47265625_{10}$$

$$\begin{matrix} 4 & 3 & 2 & 1 & 0 \\ 1 & E & 9 & A & D \end{matrix}_{16} = 1 \times 16^4 + 14 \times 16^3 + 9 \times 16^2 + 10 \times 16^1 + 13 \times 16^0 = 65536 + 57344 + 2304 + 160 + 13 = 125357_{10}$$

$$\begin{matrix} 2 & 1 & 0 & -1 & -2 \\ 4 & C & 5 & .F & B \end{matrix}_{16} = 4 \times 16^2 + 12 \times 16^1 + 5 \times 16^0 + 15 \times 16^{-1} + 11 \times 16^{-2} = 1024 + 192 + 5 + 0.9375 + 0.0429687 = 1221.9804687_{10}$$

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales (Conversión entre las distintas bases de numeración)

Conversión de decimal a cualquier otra base

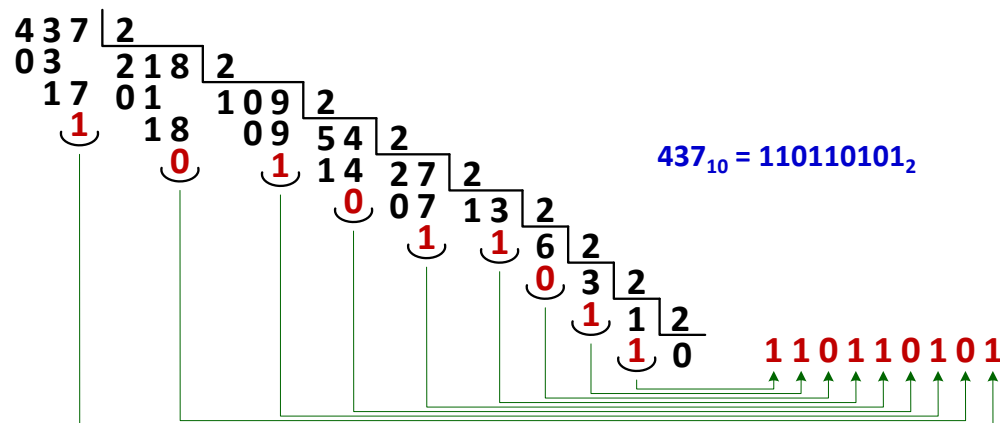
Para realizar la conversión de una cantidad expresada en decimal a cualquier otra base se emplea el **método iterativo**, cuyas principales características son las siguientes:

- Se opera usando la aritmética de la base de origen (**base 10**).
- La conversión de las partes entera y fraccionaria se realiza por separado y de forma diferente.
- El resultado final se obtiene colocando a la izquierda de la coma de decimales el resultado de la conversión de la parte entera y a la derecha el resultado de la conversión de la parte fraccionaria.

Conversión de la parte entera usando el método iterativo

- Se divide sucesivamente la parte entera por la base de destino.
- Se repite la operación hasta que el cociente sea 0.
- La representación de la parte entera en la nueva base es igual a la secuencia ordenada de los restos de todas las divisiones realizadas, en orden inverso al de su obtención, es decir, el dígito más significativo será el obtenido en último lugar.

Ejemplo: Convertir a binario el número decimal 437.



Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales (Conversión entre las distintas bases de numeración)

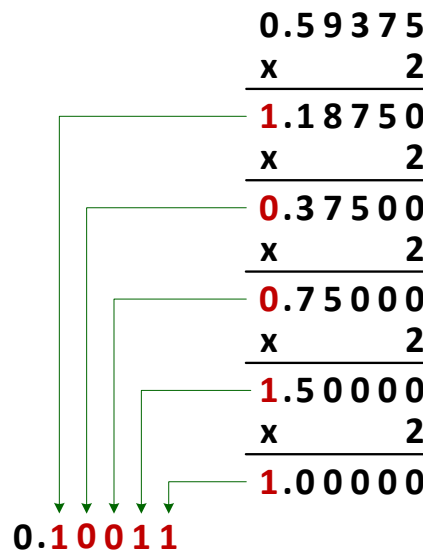
Conversión de decimal a cualquier otra base

Conversión de la parte fraccionaria usando el método iterativo

- Se multiplica sucesivamente la parte fraccionaria por la base de destino.
- Por cada multiplicación realizada se obtiene un dígito del resultado.
- Se repite la operación hasta que la parte fraccionaria del resultado sea 0, o bien, se haya obtenido el número de dígitos decimales deseado.
- La representación de la parte fraccionaria en la nueva base es igual a la secuencia ordenada de las partes enteras de los resultados de las multiplicaciones, colocadas en el mismo orden en que se han obtenido, es decir, el dígito más significativo será el obtenido primer último lugar.

Ejemplo: Convertir a binario el número decimal 0.59375.

	0.59375
	x 2
	1.18750
	x 2
	0.37500
	x 2
	0.75000
	x 2
	1.50000
	x 2
	1.00000
0.10011	



$$0.59375_{10} = 0.10011_2$$

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales (Conversión entre las distintas bases de numeración)

Conversión de binario a octal o hexadecimal y viceversa

Los sistemas de numeración octal y hexadecimal poseen una característica especial, consistente en que sus bases respectivas son potencia de 2: $8 = 2^3$ y $16 = 2^4$.

Este hecho posibilita la realización directa de conversiones de cantidades entre estos sistemas sin necesidad de realizar operaciones, como ocurre en el método general estudiado anteriormente.

Conversión de binario a octal o hexadecimal

- La conversión de las partes entera y fraccionaria se realiza simultáneamente.
- Se divide la combinación binaria de origen en grupos de 3 (octal) o 4 (hexadecimal) bits, a izquierda y derecha de la coma de decimales.
- Se sustituye cada grupo de bits por su dígito octal o hexadecimal correspondiente.

Ejemplo 1: Convertir a octal el número binario 10111.01111.

$$\begin{array}{ccccccc} \underline{010111} & . & \underline{011110} & \\ 2 & 7 & . & 3 & 6 \end{array}$$
$$10111.01111_2 = 27.36_8$$

Ejemplo 2: Convertir a hexadecimal el número binario 1011011.11111.

$$\begin{array}{ccccccc} \underline{01011011} & . & \underline{11111000} & \\ 5 & B & . & F & 8 \end{array}$$
$$1011011.11111_2 = 5B.F8_{16}$$

Tema 1. Introducción a los sistemas digitales

Sistemas de numeración posicionales (Conversión entre las distintas bases de numeración)

Conversión de binario a octal o hexadecimal y viceversa

Conversión de octal o hexadecimal a binario

- La conversión de las partes entera y fraccionaria se realiza simultáneamente.
- Se sustituye cada dígito octal o hexadecimal por la correspondiente combinación binaria de 3 o 4 bits, respectivamente.
- Se eliminan los ceros situados a la izquierda de la parte entera y a la derecha de la parte fraccionaria.

Ejemplo 1: Convertir a binario los números octales 367.46 y 123.765

$$\begin{array}{ccccccc} 3 & 6 & 7 & . & 4 & 6 & \\ \hline 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & . & 1 & 0 & 0 & 1 & 1 & 0 \end{array}$$
$$367.46_8 = 11110111.10011_2$$
$$\begin{array}{ccccccc} 1 & 2 & 3 & . & 7 & 6 & 5 \\ \hline 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & . & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \end{array}$$
$$123.765_8 = 1010011.111110101_2$$

Ejemplo 2: Convertir a binario los números hexadecimales 7CF.A3E y ABC.DEF.

$$\begin{array}{ccccccc} 7 & C & F & . & A & 3 & E \\ \hline 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & . & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \end{array}$$
$$7CF.A3E_{16} = 11111001111.101000111110_2$$
$$\begin{array}{ccccccc} A & B & C & . & D & E & F \\ \hline 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & . & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{array}$$
$$ABC.DEF_{16} = 101010111100.110111101111_2$$

Tema 1. Introducción a los sistemas digitales

Codificación

Se denomina **codificación** al establecimiento de una relación entre un alfabeto de origen y un alfabeto de destino.

En el caso de los **códigos binarios**, cada elemento del alfabeto de origen se representa mediante una combinación binaria de un número determinado de bits.

Símbolo a representar ↔ 1 1 0 1 1 0 1 1 1 0 1

MSB (*Most Significant Bit*) LSB (*Least Significant Bit*)

The diagram illustrates the binary sequence 11011011101. A green double-headed arrow points to the first '1' on the left, which is labeled 'MSB (Most Significant Bit)' in blue. A red arrow points to the final '1' on the right, which is labeled 'LSB (Least Significant Bit)' in red. The bits in the sequence are colored: the first '1' is blue, the '0' is black, the next '1' is blue, the '0' is black, the next '1' is blue, the '1' is black, the next '1' is blue, the '0' is black, the next '1' is blue, the '1' is black, and the final '1' is red.

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos BCD)

En un apartado anterior se ha comprobado como la conversión de cantidades entre el sistema decimal y el binario, y viceversa, entraña cierta dificultad, ya que conlleva la realización de una serie de operaciones (divisiones o multiplicaciones).

Para superar este inconveniente se crearon los códigos **BCD (Binary Coded Decimal)**, o códigos decimales codificados en binario, que permiten realizar la conversión a decimal de forma directa, como sucede con los códigos octal y hexadecimal.

Todos los códigos BCD constan de 10 combinaciones, que permiten representar mediante combinaciones binarias los 10 símbolos del sistema decimal (0, 1, 2, 3, 4, 5, 6, 7, 8, 9). Como hay que representar 10 símbolos diferentes y la capacidad de representación de una combinación binaria de n bits es 2^n , en los códigos BCD se emplean combinaciones de 4 bits.

Dado que mediante 4 bits se pueden generar hasta 16 combinaciones y en los códigos BCD sólo se utilizan 10, por cada dígito decimal representado en BCD se "desperdician" 6 combinaciones, debido a lo cual la representación en BCD de un número decimal requiere mayor número de bits que la representación en binario de ese mismo número.

Los códigos BCD pueden ser ponderados o no. En los **códigos BCD ponderados** existe un vector de pesos ($W_3W_2W_1W_0$), de tal modo que el valor decimal correspondiente a una combinación se obtiene sumado los pesos de aquellas posiciones cuyos bits son iguales a 1.

$$\text{Valor decimal} = B_3 \times W_3 + B_2 \times W_2 + B_1 \times W_1 + B_0 \times W_0$$

Por otra parte, los códigos BCD también pueden ser autocomplementarios.

El **complemento a nueve** de un dígito decimal D es lo que le falta a dicho dígito para sumar 9, es decir $\Rightarrow 9 - D$.

Un código BCD es **autocomplementario** cuando para representar el complemento a nueve de cualquier dígito D expresado en dicho código basta con invertir los valores 0 y 1 en la combinación correspondiente a D .

$$\begin{array}{lcl} D & \longleftrightarrow & 1\ 0\ 1\ 1 \\ 9 - D & \longleftrightarrow & 0\ 1\ 0\ 0 \end{array}$$

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos BCD)

Codigos BCD ponderados

Valor decimal	BCD Natural				BCD Aiken				BCD 5-4-2-1			
	8	4	2	1	2	4	2	1	5	4	2	1
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1	0	0	1	1
4	0	1	0	0	0	1	0	0	0	1	0	0
5	0	1	0	1	1	0	1	1	1	0	0	0
6	0	1	1	0	1	1	0	0	1	0	0	1
7	0	1	1	1	1	1	0	1	1	0	1	0
8	1	0	0	0	1	1	1	0	1	0	1	1
9	1	0	0	1	1	1	1	1	1	1	0	0

El código BCD Aiken es autocomplementario.

Ejemplos de representación de cantidades decimales mediante códigos BCD ponderados:

Decimal		BCD Natural		BCD Aiken		BCD 5-4-2-1
9 2 3 . 7	↔	1001 0010 0011 . 0111		1111 0010 0011 . 1101		1100 0010 0011 . 1010
1 8 . 6 5	↔	0001 1000 . 0110 0101		0001 1110 . 1100 1011		0001 1011 . 1001 1000
7 . 2 0 4	↔	0111 . 0010 0000 0100		1101 . 0010 0000 0100		1010 . 0010 0000 0100

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos BCD)

Codigos BCD no ponderados

El código BCD no ponderado más conocido es el código **BCD Exceso 3**.

Para obtener el código BCD Exceso 3, primero se suma 3 al valor decimal correspondiente y a continuación se representa el resultado en binario natural.

Valor decimal	BCD Exceso 3			
0	0	0	1	1
1	0	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	1	1	1
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0

El código BCD Exceso 3 también es autocomplementario.

Ejemplos de representación de cantidades decimales en código BCD Exceso 3:

Decimal		BCD Exceso 3
9 2 3 . 7	↔	1100 0101 0110 . 1010
1 8 . 6 5	↔	0100 1011 . 1001 1000
7 . 2 0 4	↔	1010 . 0101 0011 0111

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos continuos o progresivos)

Se dice que dos combinaciones binarias son **adyacentes** cuando éstas difieren en el valor de un solo bit.

Ejemplos:

11010 1 01	0 0111001	011010 1 0
11010 0 01	1 0111001	011010 0 0

Un código binario es **continuo** o **progresivo** cuando las combinaciones binarias que se emplean en dicho código para representar valores decimales consecutivos son adyacentes.

Ejemplo de código continuo:

Valor decimal	Código continuo			
0	0	0	1	1
1	1	0	1	1
2	1	0	1	0
3	1	1	1	0
4	1	1	0	0
5	0	1	0	0
6	0	1	0	1
7	0	0	0	1
8	1	0	0	1
9	1	0	0	0

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos cíclicos)

Un código binario **cíclico** es un código continuo en el cual, además, la combinación binaria empleada para representar el último valor decimal es adyacente con la utilizada para representar el primero.

Existen códigos cíclicos completos e incompletos.

Códigos cíclicos completos

Un código cíclico es **completo** si emplea combinaciones binarias de **n** bits para representar los diferentes valores decimales y posee **2ⁿ** combinaciones.

Los códigos cíclicos completos más conocidos son los códigos **Gray** o **reflejados**, denominados así porque para obtener un código de **n** bits se parte de uno de **n-1** bits, reflejando las diferentes combinaciones y añadiendo un nuevo bit a la izquierda, que será igual a **0** para la primera mitad de las combinaciones resultantes e igual a **1** para la segunda mitad.

G_1	G_0
0	0
0	1
1	1
1	0

Código Gray de 3 bits:

Valor decimal	Código Gray de 3 bits		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos cíclicos)

Códigos cíclicos incompletos

Un código cíclico es **incompleto** si emplea combinaciones binarias de **n** bits para representar los diferentes valores decimales y posee menos de **2ⁿ** combinaciones, es decir que no utiliza todas las combinaciones que se pueden formar con el número de bits empleados.

Los códigos cíclicos incompletos más conocidos son los códigos de **Johnson**, que para combinaciones de **n** bits poseen una capacidad de representación de **2n**.

A continuación se muestra un código de Johnson de 8 bits, por lo que posee 16 combinaciones.

Valor decimal	Código Johnson de 8 bits							
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	1	1
3	0	0	0	0	0	1	1	1
4	0	0	0	0	1	1	1	1
5	0	0	0	1	1	1	1	1
6	0	0	1	1	1	1	1	1
7	0	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1	0
10	1	1	1	1	1	1	0	0
11	1	1	1	1	1	0	0	0
12	1	1	1	1	0	0	0	0
13	1	1	1	0	0	0	0	0
14	1	1	0	0	0	0	0	0
15	1	0	0	0	0	0	0	0

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos detectores de errores)

Se denomina **distancia** entre dos combinaciones binarias al número de bits cuyo valor es diferente en dichas combinaciones.

<u>Distancia 1</u>	<u>Distancia 2</u>	<u>Distancia 4</u>
11010 1 01	0011 1 001	0 1 000 1 0
11010 0 01	0 1 11 0 001	0 0 10 1 000

Se denomina **distancia mínima** de un código a la menor de las distancias entre dos combinaciones cualesquiera de dicho código.

Los **códigos detectores de errores** son aquellos que permiten detectar la presencia de errores en cualquiera de sus combinaciones.

- Para que un código puede detectar errores, su distancia mínima debe ser superior a la unidad.
- De manera general, el número de errores que puede detectar un código es igual a su distancia mínima menos 1.
- Existen dos tipos de códigos detectores de errores: los códigos de paridad y los códigos de número de unos constante.

Códigos de paridad

Los **códigos de paridad** son aquellos que poseen una paridad fija en todas sus combinaciones. Si el número de unos de todas las combinaciones del código es par se dice que el código es de **paridad par**, y si es impar se dice que el código es de **paridad impar**.

Los códigos de paridad se obtienen a partir de un código de distancia mínima unidad, añadiendo un bit adicional a cada combinación, cuyo valor será el adecuado para que todas las combinaciones del código posean la paridad deseada (par o impar).

Valor decimal	B ₂	B ₁	B ₀	P
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	0
7	1	1	1	1

Código de paridad par

Valor decimal	B ₂	B ₁	B ₀	I
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Código de paridad impar

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos detectores de errores)

Códigos de número de unos constante

Los **códigos de número de unos constante** son aquellos códigos detectores de errores que poseen el mismo número de unos en todas sus combinaciones.

Los códigos de este tipo más conocidos son el **código 2 entre 5** y el **código biquinario**, que permiten representar los diez dígitos del sistema decimal mediante combinaciones de 5 y 7 bits, respectivamente.

Ambos códigos poseen 2 unos en todas las combinaciones, y al igual que los códigos de paridad poseen distancia mínima 2, por lo que permiten la detección de errores en un solo bit de la combinación.

Valor decimal	Código 2 entre 5				
	0	1	2	3	6
0	0	1	1	0	0
1	1	1	0	0	0
2	1	0	1	0	0
3	1	0	0	1	0
4	0	1	0	1	0
5	0	0	1	1	0
6	1	0	0	0	1
7	0	1	0	0	1
8	0	0	1	0	1
9	0	0	0	1	1

Valor decimal	Código biquinario						
	5	0	4	3	2	1	0
0	0	1	0	0	0	0	1
1	0	1	0	0	0	1	0
2	0	1	0	0	1	0	0
3	0	1	0	1	0	0	0
4	0	1	1	0	0	0	0
5	1	0	0	0	0	0	1
6	1	0	0	0	0	1	0
7	1	0	0	0	1	0	0
8	1	0	0	1	0	0	0
9	1	0	1	0	0	0	0

- Para detectar errores cuando se usa un código de paridad, se debe realizar una detección de la paridad contraria a la que posea dicho código.
- Para detectar errores cuando se usan los códigos 2 entre 5 o biquinario, se debe realizar una detección de paridad impar.

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos correctores de errores)

Los **códigos correctores de errores** son aquellos que no sólo permiten detectar la presencia de errores en sus combinaciones, sino que también permiten la corrección de los mismos, es decir, la recuperación de la combinación correcta original.

Para que un código pueda corregir errores su distancia mínima debe ser al menos 3.

Códigos de Hamming

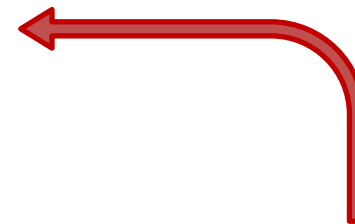
Los códigos correctores de errores más conocidos son los **códigos de Hamming**, que permiten la corrección de errores en un solo bit de la combinación.

Para la obtención de un Código de Hamming se parte de un código de distancia mínima unidad de **m** bits, al cual se le añaden **c** bits en cada combinación, con lo cual se obtiene un código de **m + c** bits.

El número de bits que deben añadirse al código original (**c**) es el menor valor entero que cumpla la siguiente expresión: $2^c \geq m + c + 1$.

Para llevar a cabo la detección y corrección de errores en el código de Hamming resultante, se realizan **c** detecciones de paridad sobre sendos grupos de bits de la combinación a chequear, cuyo resultado indica la ausencia o presencia de error en la misma, y en caso afirmativo, la localización del error de la forma en que se indica en la siguiente tabla para un código de Hamming de 7 bits:

Bit erróneo	D ₃	D ₂	D ₁
Ninguno	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1



Resultado de las detecciones de paridad

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos correctores de errores)

Para ilustrar la forma de generación de un código de Hamming, así como el funcionamiento del mismo, a continuación se va a generar un código de Hamming a partir de un código BCD Natural, que se representa en la siguiente tabla.

Valor decimal	N ₃	N ₂	N ₁	N ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

El código de partida es de 4 bits, por lo que el número de bits que se deberán añadir al código para satisfacer la expresión es $c = 3$. Así pues, en el código de Hamming resultante tendrá 7 bits ($4 + 3$) en cada combinación, que se van a denominar de la forma siguiente:

$$B_7 B_6 B_5 B_4 B_3 B_2 B_1$$

El paso siguiente consiste en decidir cuales de esos 7 bits son los cuatro que ya tenemos y cuales son los tres nuevos bits que hay que generar.

Por otro lado, como se van a añadir 3 nuevos bits al código, para la detección de errores en el código resultante se deberán realizar también 3 detecciones de paridad (D_3 , D_2 y D_1) cuyos resultados indicarán la posición del bit erróneo (en caso de existir) según el formato indicado en la siguiente tabla.

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos correctores de errores)

Bit erróneo	D ₃	D ₂	D ₁
Ninguno	0	0	0
B1	0	0	1
B2	0	1	0
B3	0	1	1
B4	1	0	0
B5	1	0	1
B6	1	1	0
B7	1	1	1

Como se puede apreciar en la tabla anterior, D₁ debe detectar el error cuando esté localizado en los bits B₁, B₃, B₅ o B₇, D₂ lo debe detectar cuando esté localizado en los bits B₂, B₃, B₆ o B₇, y D₃ lo debe detectar cuando esté localizado en los bits B₄, B₅, B₆ o B₇.

Para poder conseguir este objetivo se genera el código de tal forma que los grupos de bits **B₁B₃B₅B₇**, **B₂B₃B₆B₇** y **B₄B₅B₆B₇** tengan siempre paridad par en todas las combinaciones de dicho código. De este modo, al producirse un error en uno de los bits cambiará a impar la paridad del grupo que contenga a dicho bit y será detectado por la función correspondiente.

Dado que a los diferentes grupos de bits anteriores se les va a asignar una paridad par, las funciones D₁, D₂ y D₃ serán detecciones de paridad impar (la paridad contraria, para que proporcionen un 1 al detectar un error) y sus expresiones serán las siguientes:

$$D_1 = \mathbf{B_1} \oplus B_3 \oplus B_5 \oplus B_7$$

$$D_2 = \mathbf{B_2} \oplus B_3 \oplus B_6 \oplus B_7$$

$$D_3 = \mathbf{B_4} \oplus B_5 \oplus B_6 \oplus B_7$$

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos correctores de errores)

El modo de conseguir que los grupos de bits $B_1B_3B_5B_7$, $B_2B_3B_6B_7$ y $B_4B_5B_6B_7$ tengan siempre paridad par en todas las combinaciones del código es obtener uno de los bits a partir de los otros tres.

Como se puede apreciar en las expresiones de la diapositiva anterior, los bits B_1 , B_2 y B_4 son los únicos que aparecen en una sola función de detección, por lo que serán éstos los tres nuevos bits que se añadan al código original, mientras que los bits B_3 , B_5 , B_6 y B_7 serán los cuatro bits ya existentes en dicho código.

Los bits B_1 , B_2 y B_4 se obtendrán a partir de los cuatro existentes aplicando las siguientes funciones:

$$B_1 = B_3 \oplus B_5 \oplus B_7$$

$$B_2 = B_3 \oplus B_6 \oplus B_7$$

$$B_4 = B_5 \oplus B_6 \oplus B_7$$

Con todo ello, el código de Hamming que se obtiene a partir del código BCD natural es el que se muestra en la siguiente tabla:

Valor decimal	B_7	B_6	B_5	B_4	B_3	B_2	B_1
0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	0	1	1	0	0	1
3	0	0	1	1	1	1	0
4	0	1	0	1	0	1	0
5	0	1	0	1	1	0	1
6	0	1	1	0	0	1	1
7	0	1	1	0	1	0	0
8	1	0	0	1	0	1	1
9	1	0	0	1	1	0	0

Tema 1. Introducción a los sistemas digitales

Codificación (Códigos correctores de errores)

Por último se va a comprobar el funcionamiento del código. Supongamos que se recibe la combinación correspondiente al 4.

Valor decimal	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	0	1	1	0	0	1
3	0	0	1	1	1	1	0
4	0	1	0	1	0	1	0
5	0	1	0	1	1	0	1
6	0	1	1	0	0	1	1
7	0	1	1	0	1	0	0
8	1	0	0	1	0	1	1
9	1	0	0	1	1	0	0

Valor decimal	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
4	0	1	0	1	0	1	0

$$D_1 = B_1 \oplus B_3 \oplus B_5 \oplus B_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$D_2 = B_2 \oplus B_3 \oplus B_6 \oplus B_7 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$D_3 = B_4 \oplus B_5 \oplus B_6 \oplus B_7 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

Valor decimal	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
4	0	0	0	1	0	1	0

$$D_1 = B_1 \oplus B_3 \oplus B_5 \oplus B_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$D_2 = B_2 \oplus B_3 \oplus B_6 \oplus B_7 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$$D_3 = B_4 \oplus B_5 \oplus B_6 \oplus B_7 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

Se observa como al aplicar sobre la combinación 4 del código las tres detecciones de paridad proporcionan el resultado: $D_3D_2D_1 = 000$, lo cual significa que dicha combinación es correcta, es decir, que pertenece al código.

Sin embargo, al producirse un error en el bit B₆ de dicha combinación (pasa de 1 a 0), si se vuelven a aplicar las tres detecciones de paridad proporcionan en este caso el resultado: $D_3D_2D_1 = 110$, lo cual, según la tabla mostrada al comienzo de este apartado, significa que existe un error en el bit B₆.

Por último, una vez detectada la existencia del error y averiguada su posición, para corregirlo bastará con invertir el valor del bit correspondiente a esa posición en la combinación que se está chequeando.